



DESCRIPTION OF COURSEWORK

Course Code	CST209*
Course Name	Object-Oriented Programming-C++
Lecturer	Danial Kafi Ahmad
Academic Session	2026/04
Assessment Title	Final Project

A. Introduction/ Situation/ Background Information

The objective of this final project is to create a functional C++ program by applying object-oriented programming knowledge and other relevant algorithms to solve the problem described in Section F.

B. Course Learning Outcomes (CLO) covered

At the end of this assessment, students are able to:

CLO 3 Apply knowledge to write algorithms with Object-Oriented and C++ by using class, object, inherit, and polymorphic.

C. University Policy on Academic Misconduct

1. Academic misconduct is a serious offense in Xiamen University Malaysia. It can be defined as any of the following:
 - i. **Plagiarism** is submitting or presenting someone else's work, words, ideas, data or information as your own intentionally or unintentionally. This includes incorporating published and unpublished material, whether in manuscript, printed or electronic form into your work without acknowledging the source (the person and the work).
 - ii. **Collusion** is two or more people collaborating on a piece of work (in part or whole) which is intended to be wholly individual and passed it off as own individual work.
 - iii. **Cheating** is an act of dishonesty or fraud in order to gain an unfair advantage in an assessment. This includes using or attempting to use, or assisting another to use materials

that are prohibited or inappropriate, commissioning work from a third party, falsifying data, or breaching any examination rules.

2. All assessments submitted must be the student's own work, without any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. Any form of academic misconduct, including using prohibited materials or inappropriate assistance, is a serious offense and will result in a zero mark for the entire assessment or part of it. If there is more than one guilty party, such as in case of collusion, all parties involved will receive the same penalty.

D. Instruction to Students

1. This is an individual project.
2. Submit a softcopy (**zipped file containing project folder (cpp and header files) and your report**) of the project via Moodle. The report should be submitted in **word** format. Take note that the report should also include the complete code. *Include citations as comments in your code if you have used codes from any online or published resource.*
3. Submission deadline: **9 July 2026 before 11:59 pm**
4. ***Late submissions will receive a 0 mark.***
5. Your report should contain the following information:
 - a. Cover page with your Name and your Student Id.
 - b. Answers for the assignment task in Section F.
 - c. List of references (this includes codes taken from online or published resources - these must be cited and referenced accordingly).

E. Evaluation Breakdown

No.	Component Title	Percentage (%)
Part 1: Building an application		
a.	Classes and objects	20
b.	Inheritance	10
c.	Polymorphism	10
d.	STL/Files	10

e.	Exception handling	5
f.	Quality of implementation	15
Part 2: Project Documentation		
a.	STL Explanation	5
b.	Inheritance and polymorphism explanation	20
c.	Static and/or additional functions	5
	TOTAL	100

F. Task(s)

Part I: Building a Hospital Patient and Appointment Management Application (70 Marks)

In this project, you must apply knowledge of C++ object-oriented programming to design and implement a Hospital Patient and Appointment Management application that allows users to manage patient registrations, doctor assignments, medical appointments, and consultation billing records. Refer to the following output sample for the general overall expectation of the design and looks, and the requirement below. Take note that there are possibilities of exploring beyond the fundamentals conceptual in order to satisfy the requirement.

```
=====
                        HOSPITAL MANAGEMENT SYSTEM MENU
=====
1. Add New Patient
2. Add New Doctor
3. Add New Medical Service
4. Schedule Appointment for Patient
5. Set Appointment Status / Bill Modifier
6. View All Users (Runtime Polymorphism)
7. View Medical Services Catalog
0. Exit System
-----
Enter choice: 1

--- Add New Patient ---
Enter Patient ID:
```

Figure 1. Output sample.

The program should meet the following requirements but not limited to:

The system must handle three primary entities: **Users (Patients and Doctors)**, **Medical Services/Clinics**, and **Appointment Records (including billing/fees)**.

1. Core Class Design (Concepts of OOP, Classes, and Objects)

Design and implement the following base classes:

- **MedicalService Class:**
 - **Attributes:** serviceCode (string), serviceName (string), baseFee (double).
 - **Methods:** Implement at least two overloaded constructors to demonstrate polymorphism (e.g., one taking all parameters, one defaulting baseFee to a standard consultation rate of 50.0), and a method to display service details.
- **AppointmentRecord Class (Composition):**
 - This class represents the relationship between a Patient, a Medical Service, and the final status/billing outcome.
 - **Attributes:** A private MedicalService object (demonstrates Composition – *AppointmentRecord has a MedicalService*), and a string appointmentStatus (e.g., "Scheduled", "Completed", "Cancelled").
 - **Methods:** Getters for the service code and appointment status, and a method to calculate and return the total billing amount (e.g., applying standard modifiers, insurance coverage tiers, or emergency surcharges).

2. Inheritance and Abstract Concepts (Inheritance & Polymorphism)

To manage different user types within the hospital effectively, you must establish an inheritance hierarchy.

- **Person (Abstract Base Class - ABC):**
 - **Attributes:** name (string), id (string).
 - **Methods:** A base constructor, and a pure virtual function (= 0) for displaying the specific institutional role of the person: virtual void displayRole() const = 0;
- **Patient Class (Derived Class):**
 - **Inheritance:** Must inherit from Person.
 - **Attributes:** insuranceProvider (string), and a collection (e.g., std::vector) of AppointmentRecord objects.
 - **Methods:**

override displayRole() const: Overrides the pure virtual function to display the patient's details and active appointments

scheduleAppointment(MedicalService s): Adds a new AppointmentRecord instance into the patient's collection (with a default status of "Scheduled").

setAppointmentStatus(std::string code, std::string status): Searches for the specific appointment record matching the given service code and updates its status.

calculateTotalMedicalExpenses(): Calculates and returns the patient's current cumulative medical bills based on their scheduled or completed services.

- **Doctor Class (Derived Class):**

- **Inheritance:** Must inherit from Person.
- **Attributes:** specializationDepartment (string), assignedClinicCode (string).
- **Methods:** Override the displayRole() function. assignToClinic(string code): Assigns a specialized clinic or department to the doctor.

3. System Management and Dynamic Use (Polymorphism)

- **Main Menu:** Create a complete, user-friendly menu in the main() function to navigate the system (Add Patient, Add Doctor, Add Medical Service, Schedule Appointment, Set Status/Billing, View All Users, etc.).
- **User Collection:** Create a single container (std::vector<Person*>) to store both Patient and Doctor objects using dynamic memory allocation (new).
- **Polymorphic Display:** Implement a "View All Users" option that iterates through the std::vector<Person*> container and calls the displayRole() method for each object. This must showcase **runtime polymorphism**.
- **Memory Management:** Ensure all dynamically allocated objects (Person* pointers) are properly deleted at the end of the program to prevent memory leaks.

Part II: Project Documentation (30 Marks)

Create project documentation that covers the following details:

1. Explain why the STLs you chose to use to develop this hospital application are structurally suitable based on this project context.
2. Explain the rationale behind the inheritance and polymorphism you implemented in your healthcare system project. Clarify why a class should inherit from another class (e.g., Doctor/Patient from Person) and how you achieved runtime polymorphism.

- Explain the rationale for using a static function in your program (e.g., tracking total patient count or generating auto-incremented tracking IDs) and explain any additional elements or advanced components that you integrated into your code.

APPENDIX 1

MARKING RUBRICS

Component Title	Part 1: Application					Percentage (%)	35
Criteria	Score and Descriptors					Weight (%)	Marks
	Excellent (5)	Good (4)	Average (3)	Need Improvement (2)	Poor (1)		
Classes and Objects	Classes are well organised and implemented with multiple header files and if necessary multiple CPP files. Additional functions demonstrate OO capabilities.	Classes and objects are fully implemented with the separation of class specification implementation. Additional functions have some OO capabilities.	Classes and objects are fully implemented. Additional functions relevant but lacking in OO capabilities.	Partial Implementation of classes and objects. No additional functions or little relevance.	Classes and object were not implemented or is not used at all. No additional functions.	20	
Inheritance	Excellent implementation of Inheritance with base classes and several derived classes.	Adequate implementation of inheritance.	Minimal inheritance implemented.	Partial inheritance implemented.	No inheritance seen in the code.	10	
Polymorphism	Excellent implementation of polymorphism with abstract classes.	Adequate implementation of polymorphism that covers function overloading and operator overloading.	Minimal implementation of polymorphism with some function and operator overloading.	Partial implementation of polymorphism with some function and operator overloading.	No polymorphism implemented.	10	
STL/Files	Excellent STL implementation and possibly self-developed template	Adequate use of STL/ Adequate use of files for storage. Data	Minimal use of STL/ Minimal use of files for storage. All required data are stored and	Partial or improper use of STL/ Partial use of files for storage. Not all	No STL used/ No file storage implemented.	10	

	libraries/ Excellent implementation of file usage for data storage and retrieval. This includes searching and retrieving algorithm in relation to extensive data.	and configuration. information is also stored and retrieved from files or etc.	retrieved from files or etc.	data are stored retrieved from file etc.			
Exception Handling	Excellent exception handling with all errors caught and handled appropriately.	Adequate exception handling by handling all important errors as well as recovering from some of those errors.	Minimal exception handling where important errors are handled by printing out proper error statements.	Partial exception handling where error is acknowledged and program exits.	No exception handling implemented.	5	
Quality of Implementation	Excellent coding format and standards with intuitive UI. Files are named appropriately. Development uses project files for multi- file source code.	Program runs very well with good UI. Code conforms to coding standards and format.	Program runs perfectly with simple UI.	Program can compile and run but crashes.	Program cannot compile and/or run.	15	
TOTAL (Part I)						70	
Component Title	Part II: Project Documentation				Percentage (%)	15	
Criteria	Score and Descriptors					Weight (%)	Marks
	Excellent (5)	Good (4)	Average (3)	Need Improvement (2)	Poor (1)		
STL Explanation	The explanations are clear and concise to address the question accurately.	The explanations are good but missing a few essential points.	The explanations are acceptable but with a few inaccurate statements.	The explanations are vague and contain several inaccurate statements.	The explanations are almost irrelevant or wrong or no attempt	5	

Inheritance and polymorphism explanation	The explanations are clear and concise to address the question accurately.	The explanations are good but missing a few essential points.	The explanations are acceptable but with a few inaccurate statements.	The explanations are vague and contain several inaccurate statements.	The explanations are almost irrelevant or wrong or no attempt.	20	
Static and/or additional functions	Static usage is explained in detail. Additional functions thoroughly explained and highly relevant. Adds value to the solution.	Static usage is well-explained. Additional functions explained. Its relevance is clear.	Static explained with some gaps. Additional functions explained but needs more details.	Static usage not clear. Additional functions not clearly explained.	No static and/or additional functions.	5	
TOTAL (Part II)						30	

-End of Document-